

Bases de Données Avancées

Rapport de Travaux Pratiques
& Travaux Dirigés

Réalisé par : EL ABBASSI Mohamed Taha

Encadrant : Prof. Hrimech

Année Universitaire 2025 – 2026

Contents

1	TD 2 — Sous-requêtes SQL	4
	Présentation	4
1.1	Exercice 1 — Base de données ETUDIANTS	4
1.1.1	Q1 — Étudiants dont la date de naissance est inconnue	4
1.1.2	Q2 — Étudiants dont la 2 ^e lettre du nom est 'a'	4
1.1.3	Q3 — Matières avec coefficient entre 1 et 2	4
1.1.4	Q4 — Notes de l'étudiant n°1 égales aux notes de l'étudiant n°2	4
1.1.5	Q5 — Notes de l'étudiant n°1 supérieures aux notes de l'étudiant n°2	4
1.1.6	Q6 — Notes de l'étudiant n°1 inférieures à toutes les notes de l'étudiant n°9	5
1.1.7	Q7 — Étudiants sans aucune note	5
1.1.8	Q8 — Âge moyen des garçons et des filles au 1 ^{er} janvier 2000	5
1.1.9	Q9 — Nom et grade des enseignants d'histoire	5
1.1.10	Q10 — Étudiants sans notes en Sociologie	5
1.2	Exercice 2 — Base de données EMP/DEPT	5
1.2.1	Q11 — Lieux des départements avec commission (sous-requête)	5
1.2.2	Q12 — Départements avec au moins un ingénieur	6
1.2.3	Q13 — Départements sans ingénieur	6
1.2.4	Q14 — Employés gagnant moins de 50% du salaire de leur supérieur	6
1.2.5	Q15 — Employés gagnant plus que tous les commerciaux	6
1.2.6	Q16 — Employés gagnant plus que tous les commerciaux de leur département	6
1.2.7	Q17 — Même poste et même supérieur que BERGER (multi-colonnes)	7
2	TP — Vues et Index	8
	Présentation	8
2.1	Exercice 1 — Vues	8
2.1.1	Q1 — Création de la vue V_EMP	8
2.1.2	Q2 — Sélection avec GAINS > 10 000	8
2.1.3	Q3 — Mise à jour du nom MARTIN via la vue	8
2.1.4	Q4 — Vue VEMP10 sans CHECK + insertion hors périmètre	8
2.1.5	Q5 — Suppression et recréation avec WITH CHECK OPTION	9
2.1.6	Q6 — Tentatives violant WITH CHECK OPTION	9
2.1.7	Q7 — Pourcentage du salaire par rapport au total département	9
2.2	Exercice 2 — Index	9
2.2.1	Q1 — Création des index	9
2.2.2	Q2 — Suppression de l'index sur NomE	9
3	TP — Les Vues en Bases de Données (Approfondi)	10

Présentation	10
3.1 Exercice 1 — Vue simple	10
3.2 Exercice 2 — Vue avec agrégations	10
3.3 Exercice 3 — WITH CHECK OPTION	10
3.4 Exercice 4 — WITH READ ONLY	11
3.5 Exercice 5 — Sécurité et abstraction	11
3.6 Exercice 6 — Analyse globale et réflexion	11
3.6.1 Architecture basée sur les vues	11
3.6.2 Limites des vues classiques	11
3.6.3 Vues matérialisées	11
3.6.4 Vues vs GRANT/REVOKE	11
3.6.5 Stratégie d'optimisation	12
4 TP PL/SQL — Variables et Structures de Données	13
Présentation	13
4.1 Exercice 1 — Variables scalaires	13
4.2 Exercice 2 — Portée des variables	13
4.3 Exercice 3 — Attribut %TYPE	13
4.4 Exercice 4 — Attribut %ROWTYPE	14
4.5 Exercice 5 — Table associative (collection)	14
4.6 Exercice intégré — RECORD + Collection + %ROWTYPE	14
4.7 Exercice final de synthèse	15
5 TP PL/SQL — Interaction BD & Structures de Contrôle	16
Présentation	16
5.1 Exercice 1 — Insérer les nombres de 1 à 100	16
5.2 Exercice 2 — Somme de 1 à 1000	16
5.3 Exercice 3 — Nombres pairs entre 1 et 50	16
5.4 Exercice 4 — Factorielle	17
5.5 Exercice 5 — Insérer 10 étudiants avec notes	17
5.6 Exercice 6 — Décompte de 10 à 1 avec WHILE	17
5.7 Exercice 7 — Augmenter le prix de tous les produits de 10%	17
5.8 Exercice 8 — Supprimer les produits dont le prix < 50	17
5.9 Exercice 9 — Tester le signe d'un nombre	18
5.10 Exercice 10 — Carrés de 1 à 20	18
6 TP PL/SQL — Curseurs	19
Présentation	19
6.1 Exercice 1 — Base Usine (curseur explicite)	19
6.1.1 Q1 — Liste de toutes les commandes	19
6.1.2 Q2 — Commandes dont le montant est supérieur à la moyenne	19
6.1.3 Q3 — Commandes avec fournisseur parisien	20
6.2 Exercice 2 — Base Gavasoftware (boucle FOR)	20
6.2.1 Q1 — Employés avec commission, par ordre décroissant	20
6.2.2 Q2 — Embauches depuis le 01/09/2006	20
6.2.3 Q3 — Employés travaillant à Créteil	20
6.2.4 Q4 — Subordonnés de Guimezas	21
6.2.5 Q5 — Moyenne des salaires	21
6.2.6 Q6 — Nombre de commissions non NULL	21

6.2.7	Q7 — Employés gagnant plus que la moyenne	21
7	TP — Procédures Stockées et Fonctions	22
	Présentation	22
7.1	Exercice 1 — Premier contact	22
7.2	Exercice 2 — Maximum de deux nombres	22
7.3	Exercice 3 — Permutation	22
7.4	Exercice 4 — Augmenter le salaire	22
7.5	Exercice 5 — Conversion Dirhams vers Euros	23
7.6	Exercice 6 — Employés au-dessus d'un seuil	23
7.7	Exercice 7 — Salaire moyen par département	23
7.8	Exercice 8 — Mise à jour d'adresse	24
7.9	Exercice 9 — Nombre d'employés par département	24
7.10	Exercice 10 — Suppression des petits salaires	24
7.11	Synthèse — Procédure vs Fonction	25
8	TP — Triggers PL/SQL	26
	Présentation	26
8.1	Création des tables	26
8.2	Tableau des règles et codes d'erreur	27
8.3	Règle 1 — Interdiction de modifier note_min dans prerequis	27
8.4	Règle 2 — Contrôle de effecMax lors d'une inscription	27
8.5	Règle 3 — Examen uniquement si le module a des inscrits	27
8.6	Règle 4 — Date d'examen postérieure à la date d'inscription	28
8.7	Règle 5 — Détection de circuit dans les prerequis	28
8.8	Règle 6 — Vérification des prerequis à l'inscription	29
8.9	Règle 7 — Mise à jour automatique de la moyenne	29
8.10	Règle 8 — Modification de note_min si elle invalide des inscriptions	29
8.11	Règle 9 — Modification de effecMax sans invalider des inscriptions	30
8.12	Exercices supplémentaires	30
8.12.1	Contrainte 10 — Module déjà validé	30
8.12.2	Contrainte 11 — Conflit de planning (7 jours)	31
8.12.3	Contrainte 12 — Moyenne calculée seulement si tous les examens sont passés	31
8.12.4	Contrainte 13 — Interdire la suppression d'un module prerequis	32
	Conclusion	33

Chapter 1

TD 2 — Sous-requêtes SQL

Présentation

Ce TD porte sur l'utilisation des sous-requêtes SQL (simples, synchronisées et multi-colonnes) sur deux bases : la base **ETUDIANTS** et la base **EMP/DEPT**.

1.1 Exercice 1 — Base de données ETUDIANTS

Schéma utilisé :

- ETUDIANT(Numetu, Nometu, Dtnaiss, Cdsexe)
- SEXE(Cdsexe, Lbsexe)
- ENSEIGNANT(Numens, Nomens, Grade, Ancien)
- MATIERE(Numat, Nomat, Coeff, Numens)
- NOTES(Numetu, Numat, Note)

1.1.1 Q1 — Étudiants dont la date de naissance est inconnue

```
1 SELECT Numetu, Nometu, Cdsexe
2 FROM ETUDIANT
3 WHERE Dtnaiss IS NULL;
```

1.1.2 Q2 — Étudiants dont la 2^e lettre du nom est 'a'

```
1 SELECT Nometu, Dtnaiss
2 FROM ETUDIANT
3 WHERE Nometu LIKE '_a%';
```

1.1.3 Q3 — Matières avec coefficient entre 1 et 2

```
1 SELECT Nomat
2 FROM MATIERE
3 WHERE Coeff BETWEEN 1 AND 2;
```

1.1.4 Q4 — Notes de l'étudiant n°1 égales aux notes de l'étudiant n°2

```
1 SELECT Note
2 FROM NOTES
3 WHERE Numetu = 1
4 AND Note IN (SELECT Note FROM NOTES WHERE Numetu = 2);
```

1.1.5 Q5 — Notes de l'étudiant n°1 supérieures aux notes de l'étudiant n°2

```
1 SELECT Note
```

```

2 FROM NOTES
3 WHERE Numetu = 1
4 AND Note > ANY (SELECT Note FROM NOTES WHERE Numetu = 2);

```

1.1.6 Q6 — Notes de l'étudiant n°1 inférieures à toutes les notes de l'étudiant n°9

```

1 SELECT Note
2 FROM NOTES
3 WHERE Numetu = 1
4 AND Note < ALL (SELECT Note FROM NOTES WHERE Numetu = 9);

```

1.1.7 Q7 — Étudiants sans aucune note

```

1 SELECT *
2 FROM ETUDIANT
3 WHERE Numetu NOT IN (SELECT Numetu FROM NOTES);

```

1.1.8 Q8 — Âge moyen des garçons et des filles au 1^{er} janvier 2000

```

1 SELECT Cdsexe,
2        AVG(YEAR('2000-01-01') - YEAR(Dtnaiss)) AS AgeMoyen
3 FROM ETUDIANT
4 WHERE Dtnaiss IS NOT NULL
5 GROUP BY Cdsexe;

```

1.1.9 Q9 — Nom et grade des enseignants d'histoire

```

1 SELECT Nomens, Grade
2 FROM ENSEIGNANT
3 WHERE Numens IN (SELECT Numens FROM MATIERE WHERE Nomat = 'Histoire');

```

1.1.10 Q10 — Étudiants sans notes en Sociologie

```

1 SELECT Numetu, Nometu
2 FROM ETUDIANT
3 WHERE Numetu NOT IN (
4     SELECT N.Numetu
5     FROM NOTES N
6     JOIN MATIERE M ON N.Numat = M.Numat
7     WHERE M.Nomat = 'Sociologie'
8 );

```

1.2 Exercice 2 — Base de données EMP/DEPT

Schéma utilisé :

- EMP (Matr, NomE, Poste, DatEmb, Sup, Salaire, Commission, NumDept)
- DEPT (NumDept, NomDept, Lieu)
- PROJET (CodeP, NomP)
- PARTICIPATION (Matr, CodeP, Fonction)

1.2.1 Q11 — Lieux des départements avec commission (sous-requête)

```
1 SELECT Lieu
2 FROM DEPT
3 WHERE NumDept IN (
4     SELECT NumDept FROM EMP WHERE Commission IS NOT NULL
5 );
```

1.2.2 Q12 — Départements avec au moins un ingénieur

Avec sous-requête :

```
1 SELECT NomDept, Lieu
2 FROM DEPT
3 WHERE NumDept IN (SELECT NumDept FROM EMP WHERE Poste = 'Ingenieur');
```

Avec jointure (possible, mais peut dupliquer des lignes) :

```
1 SELECT DISTINCT D.NomDept, D.Lieu
2 FROM DEPT D
3 JOIN EMP E ON D.NumDept = E.NumDept
4 WHERE E.Poste = 'Ingenieur';
```

1.2.3 Q13 — Départements sans ingénieur

```
1 SELECT NomDept, Lieu
2 FROM DEPT
3 WHERE NumDept NOT IN (SELECT NumDept FROM EMP WHERE Poste =
    'Ingenieur');
```

1.2.4 Q14 — Employés gagnant moins de 50% du salaire de leur supérieur

```
1 SELECT E.NomE, E.Salaire
2 FROM EMP E
3 WHERE E.Salaire < 0.5 * (
4     SELECT S.Salaire FROM EMP S WHERE S.Matr = E.Sup
5 );
```

1.2.5 Q15 — Employés gagnant plus que tous les commerciaux

```
1 SELECT NomE
2 FROM EMP
3 WHERE Salaire > ALL (SELECT Salaire FROM EMP WHERE Poste =
    'Commercial');
```

1.2.6 Q16 — Employés gagnant plus que tous les commerciaux de leur département

Solution incluant les départements sans commerciaux :

```
1 SELECT NomE
2 FROM EMP E
3 WHERE E.Salaire > ALL (
4     SELECT Salaire FROM EMP
5     WHERE Poste = 'Commercial' AND NumDept = E.NumDept
6 );
```

> ALL sur un ensemble vide retourne TRUE : les employés dans un département sans commerciaux sont inclus (logiquement valide).

Solution excluant les départements sans commerciaux :

```
1 SELECT NomE
2 FROM   EMP E
3 WHERE  E.NumDept IN (SELECT NumDept FROM EMP WHERE Poste = 'Commercial')
4 AND    E.Salaire > ALL (
5     SELECT Salaire FROM EMP
6     WHERE  Poste = 'Commercial' AND NumDept = E.NumDept
7 );
```

1.2.7 Q17 — Même poste et même supérieur que BERGER (multi-colonnes)

```
1 SELECT NomE
2 FROM   EMP
3 WHERE  (Poste, Sup) IN (SELECT Poste, Sup FROM EMP WHERE NomE =
4     'BERGER')
5 AND    NomE <> 'BERGER';
```

Chapter 2

TP — Vues et Index

Présentation

Ce TP traite de la création et de la manipulation des vues SQL (simples, avec WITH CHECK OPTION, en lecture seule) ainsi que de la gestion des index sur la base **EMP/DEPT**.

2.1 Exercice 1 — Vues

2.1.1 Q1 — Création de la vue V_EMP

```
1 CREATE VIEW V_EMP AS
2 SELECT E.Matr,
3        E.NomE,
4        E.NumDept,
5        (E.Salaire + COALESCE(E.Commission, 0)) AS GAINS,
6        D.Lieu
7 FROM   EMP E
8 JOIN   DEPT D ON E.NumDept = D.NumDept;
```

2.1.2 Q2 — Sélection avec GAINS > 10 000

```
1 SELECT * FROM V_EMP WHERE GAINS > 10000;
```

2.1.3 Q3 — Mise à jour du nom MARTIN via la vue

```
1 UPDATE V_EMP SET NomE = 'MARTIN_NOUVEAU' WHERE NomE = 'MARTIN';
```

Une vue portant sur une jointure est généralement en **lecture seule** : Oracle peut accepter cet UPDATE si la colonne modifiée provient d'une seule table de base, mais d'autres SGBD le refuseront.

2.1.4 Q4 — Vue VEMP10 sans CHECK + insertion hors périmètre

```
1 CREATE VIEW VEMP10 AS
2 SELECT * FROM EMP WHERE NumDept = 10;
3
4 -- Insertion d'un employe du departement 20
5 INSERT INTO VEMP10
6   (Matr, NomE, Poste, DatEmb, Sup, Salaire, Commission, NumDept)
7 VALUES (999, 'SOULIER', 'Technicien', '2024-01-01', NULL, 5000, NULL, 20);
8
9 -- Invisible via la vue, visible dans la table
```

```

9 SELECT * FROM VEMP10 WHERE NomE = 'SOULIER'; -- 0 ligne
10 SELECT * FROM EMP WHERE NomE = 'SOULIER'; -- 1 ligne

```

2.1.5 Q5 — Suppression et recréation avec WITH CHECK OPTION

```

1 DROP VIEW VEMP10;
2
3 CREATE VIEW VEMP10 AS
4 SELECT * FROM EMP WHERE NumDept = 10
5 WITH CHECK OPTION;

```

2.1.6 Q6 — Tentatives violant WITH CHECK OPTION

```

1 -- Insertion refusee (NumDept = 30 <> 10)
2 INSERT INTO VEMP10
3   (Matr, NomE, Poste, DatEmb, Sup, Salaire, Commission, NumDept)
4   VALUES (998, 'BALARD', 'Analyste', '2024-01-01', NULL, 6000, NULL, 30);
5
6 -- Modification de departement refusee
7 UPDATE VEMP10 SET NumDept = 20 WHERE NomE = 'DUPONT';

```

2.1.7 Q7 — Pourcentage du salaire par rapport au total département

Avec une vue :

```

1 CREATE VIEW TOTAL_DEPT AS
2 SELECT NumDept, SUM(Salaire) AS TotalSalaire
3 FROM EMP GROUP BY NumDept;
4
5 SELECT E.NomE, E.NumDept, E.Salaire,
6        ROUND(E.Salaire * 100.0 / T.TotalSalaire, 2) AS Pourcentage
7 FROM EMP E JOIN TOTAL_DEPT T ON E.NumDept = T.NumDept;

```

Sans vue (sous-requête dans FROM) :

```

1 SELECT E.NomE, E.NumDept, E.Salaire,
2        ROUND(E.Salaire * 100.0 / T.TotalSalaire, 2) AS Pourcentage
3 FROM EMP E
4 JOIN (SELECT NumDept, SUM(Salaire) AS TotalSalaire
5       FROM EMP GROUP BY NumDept) T ON E.NumDept = T.NumDept;

```

2.2 Exercice 2 — Index

2.2.1 Q1 — Création des index

```

1 -- Index unique sur NomE
2 CREATE UNIQUE INDEX idx_nomE ON EMP (NomE);
3
4 -- Index simple sur NumDept
5 CREATE INDEX idx_dept ON EMP (NumDept);

```

2.2.2 Q2 — Suppression de l'index sur NomE

```

1 DROP INDEX idx_nomE ON EMP; -- MySQL
2 -- DROP INDEX idx_nomE;    -- Oracle

```

Chapter 3

TP — Les Vues en Bases de Données (Approfondi)

Présentation

Ce TP complète le précédent en approfondissant les notions de vues simples, complexes, avec WITH CHECK OPTION, en lecture seule, et en abordant les questions de sécurité, de performance et d'architecture.

3.1 Exercice 1 — Vue simple

```
1 CREATE VIEW emp_paris AS
2 SELECT e.emp_id, e.nom, e.salaire
3 FROM   Employe e
4 JOIN   Departement d ON e.dept_id = d.dept_id
5 WHERE  d.ville = 'Paris';
6
7 SELECT * FROM emp_paris;
```

3.2 Exercice 2 — Vue avec agrégations

```
1 CREATE VIEW stat_dept AS
2 SELECT d.nom_dept,
3        MIN(e.salaire) AS salaire_min,
4        MAX(e.salaire) AS salaire_max,
5        AVG(e.salaire) AS salaire_moyen
6 FROM   Employe e
7 JOIN   Departement d ON e.dept_id = d.dept_id
8 GROUP BY d.nom_dept;
```

Cette vue est en **lecture seule** car elle contient un GROUP BY.

3.3 Exercice 3 — WITH CHECK OPTION

```
1 CREATE VIEW emp_lyon AS
2 SELECT e.*
3 FROM   Employe e
4 JOIN   Departement d ON e.dept_id = d.dept_id
5 WHERE  d.ville = 'Lyon'
6 WITH CHECK OPTION;
```

3.4 Exercice 4 — WITH READ ONLY

```
1 CREATE VIEW emp_readonly AS
2 SELECT * FROM Employe
3 WITH READ ONLY;
```

3.5 Exercice 5 — Sécurité et abstraction

```
1 -- Vue publique sans le salaire
2 CREATE VIEW emp_public AS
3 SELECT emp_id, nom, dept_id FROM Employe;
```

Une vue améliore-t-elle les performances ? Non en général : une vue classique réexécute la requête à chaque appel. Seule une **vue matérialisée** stocke physiquement les résultats et améliore les temps de lecture.

3.6 Exercice 6 — Analyse globale et réflexion

3.6.1 Architecture basée sur les vues

Profil	Vue	Contenu
Manager	vue_manager_deptX	Employés de son département
RH	Accès direct aux tables	Toutes les informations
Employé	emp_public	Sans la colonne salaire

3.6.2 Limites des vues classiques

- Réexécution complète de la requête à chaque appel.
- Pas d'index propre à la vue.
- Jointures lourdes sur des millions de lignes.

3.6.3 Vues matérialisées

```
1 CREATE MATERIALIZED VIEW stat_dept_mat
2 REFRESH COMPLETE ON DEMAND AS
3 SELECT d.nom_dept,
4        MIN(e.salaire) AS salaire_min,
5        MAX(e.salaire) AS salaire_max,
6        AVG(e.salaire) AS salaire_moyen
7 FROM   Employe e
8 JOIN   Departement d ON e.dept_id = d.dept_id
9 GROUP BY d.nom_dept;
```

3.6.4 Vues vs GRANT/REVOKE

Critère	Vues	GRANT/REVOKE
Granularité	Colonne et ligne (WHERE)	Table ou colonne entière
Flexibilité	Très fine (jointures)	Limitée aux objets
Maintenance	Recréation si schéma change	Simple
Complémentarité	Les deux se combinent idéalement	

3.6.5 Stratégie d'optimisation

- **Index** : sur `dept_id`, `ville`, etc.
- **Partitionnement** : par département ou site géographique.
- **Vues matérialisées** : pour les agrégations lourdes consultées fréquemment.
- **Cache applicatif** : pour les données peu volatiles.

Chapter 4

TP PL/SQL — Variables et Structures de Données

Présentation

Ce TP introduit les concepts fondamentaux de PL/SQL : variables scalaires, portée, attributs %TYPE et %ROWTYPE, enregistrements (RECORD) et collections (tables associatives).

4.1 Exercice 1 — Variables scalaires

```
1 SET SERVEROUTPUT ON;
2 DECLARE
3     v_nom          VARCHAR2(30) := 'Alice';
4     v_salaire       NUMBER(8,2) := 3500.50;
5     v_date_emb      DATE         := SYSDATE;
6     c_taux_impot    CONSTANT NUMBER(3,2) := 0.15;
7     v_prime         NUMBER(8,2);
8 BEGIN
9     v_prime := v_salaire * c_taux_impot;
10    DBMS_OUTPUT.PUT_LINE('Employe : ' || v_nom);
11    DBMS_OUTPUT.PUT_LINE('Prime   : ' || v_prime);
12 END;
13 /
```

4.2 Exercice 2 — Portée des variables

```
1 DECLARE
2     v_externe NUMBER := 10;
3 BEGIN
4     DECLARE
5         v_interne NUMBER := 20;
6     BEGIN
7         DBMS_OUTPUT.PUT_LINE('Interne : ' || v_interne);
8         DBMS_OUTPUT.PUT_LINE('Externe : ' || v_externe);
9     END;
10    -- v_interne inaccessible ici
11 END;
12 /
```

4.3 Exercice 3 — Attribut %TYPE

```

1 DECLARE
2     v_nom      employes.nom%TYPE;
3     v_salaire  employes.salaire%TYPE;
4 BEGIN
5     v_nom := 'Jean';  v_salaire := 5000;
6     DBMS_OUTPUT.PUT_LINE(v_nom || ' gagne ' || v_salaire);
7 END;
8 /

```

4.4 Exercice 4 — Attribut %ROWTYPE

```

1 DECLARE
2     v_emp employes%ROWTYPE;
3 BEGIN
4     v_emp.id := 1;  v_emp.nom := 'Maria';  v_emp.salaire := 6000;
5     DBMS_OUTPUT.PUT_LINE(v_emp.nom || ' - ' || v_emp.salaire);
6 END;
7 /

```

4.5 Exercice 5 — Table associative (collection)

```

1 DECLARE
2     TYPE table_salaires IS TABLE OF NUMBER INDEX BY BINARY_INTEGER;
3     v_salaires table_salaires;
4 BEGIN
5     v_salaires(1) := 3000;
6     v_salaires(2) := 4500;
7     v_salaires(3) := 5000;
8     FOR i IN 1..3 LOOP
9         DBMS_OUTPUT.PUT_LINE('Salaire [' || i || '] : ' ||
10                                v_salaires(i));
11     END LOOP;
12 END;
13 /

```

4.6 Exercice intégré — RECORD + Collection + %ROWTYPE

```

1 DECLARE
2     TYPE table_emp IS TABLE OF employes%ROWTYPE INDEX BY BINARY_INTEGER;
3     v_emps table_emp;
4 BEGIN
5     FOR i IN 1..3 LOOP
6         v_emps(i).id      := i;
7         v_emps(i).nom     := 'Employe_' || i;
8         v_emps(i).salaire := 3000 + i * 1000;
9     END LOOP;
10    FOR i IN 1..3 LOOP
11        DBMS_OUTPUT.PUT_LINE(v_emps(i).nom || ' gagne ' ||
12                                v_emps(i).salaire);
13    END LOOP;
14 END;
15 /

```

4.7 Exercice final de synthèse

```
1 DECLARE
2     TYPE type_employe IS RECORD (
3         id NUMBER, nom VARCHAR2(50), salaire NUMBER(8,2));
4     TYPE table_employes IS TABLE OF type_employe INDEX BY
5         BINARY_INTEGER;
6     v_liste table_employes;
7     v_total NUMBER(10,2) := 0;
8     v_max NUMBER(10,2) := 0;
9     v_moyenne NUMBER(10,2);
10 BEGIN
11     v_liste(1).id:=1; v_liste(1).nom:='Alice';
12     v_liste(1).salaire:=3200;
13     v_liste(2).id:=2; v_liste(2).nom:='Bob';
14     v_liste(2).salaire:=4800;
15     v_liste(3).id:=3; v_liste(3).nom:='Clara';
16     v_liste(3).salaire:=5500;
17     v_liste(4).id:=4; v_liste(4).nom:='David';
18     v_liste(4).salaire:=2900;
19     v_liste(5).id:=5; v_liste(5).nom:='Emma';
20     v_liste(5).salaire:=6100;
21     FOR i IN 1..5 LOOP
22         DBMS_OUTPUT.PUT_LINE(v_liste(i).nom || ' : ' ||
23             v_liste(i).salaire);
24         v_total := v_total + v_liste(i).salaire;
25         IF v_liste(i).salaire > v_max THEN v_max := v_liste(i).salaire;
26         END IF;
27     END LOOP;
28     v_moyenne := v_total / 5;
29     DBMS_OUTPUT.PUT_LINE('Moyenne : ' || v_moyenne);
30     DBMS_OUTPUT.PUT_LINE('Maximum : ' || v_max);
31 END;
```

Chapter 5

TP PL/SQL — Interaction BD & Structures de Contrôle

Présentation

Ce TP couvre les interactions entre PL/SQL et la base de données (SELECT INTO, INSERT, UPDATE, DELETE) ainsi que les structures de contrôle (IF, LOOP, WHILE, FOR).

5.1 Exercice 1 — Insérer les nombres de 1 à 100

```
1 CREATE TABLE Nombre (n NUMBER);
2
3 BEGIN
4     FOR i IN 1..100 LOOP
5         INSERT INTO Nombre (n) VALUES (i);
6     END LOOP;
7     COMMIT;
8     DBMS_OUTPUT.PUT_LINE('100 nombres inseres.');
```

5.2 Exercice 2 — Somme de 1 à 1000

```
1 DECLARE
2     v_somme NUMBER := 0;
3 BEGIN
4     FOR i IN 1..1000 LOOP v_somme := v_somme + i; END LOOP;
5     DBMS_OUTPUT.PUT_LINE('Somme = ' || v_somme); -- 500500
6 END;
7 /
```

5.3 Exercice 3 — Nombres pairs entre 1 et 50

```
1 BEGIN
2     FOR i IN 1..50 LOOP
3         IF MOD(i, 2) = 0 THEN DBMS_OUTPUT.PUT_LINE(i); END IF;
4     END LOOP;
5 END;
6 /
```

5.4 Exercice 4 — Factorielle

```

1 DECLARE
2     v_n          NUMBER := 7;
3     v_factoriel  NUMBER := 1;
4 BEGIN
5     FOR i IN 1..v_n LOOP v_factoriel := v_factoriel * i; END LOOP;
6     DBMS_OUTPUT.PUT_LINE(v_n || ' ! = ' || v_factoriel);  -- 5040
7 END;
8 /

```

5.5 Exercice 5 — Insérer 10 étudiants avec notes

```

1 CREATE TABLE Etudiant (id NUMBER PRIMARY KEY, nom VARCHAR2(50),
2                          note NUMBER(4,2));
3 BEGIN
4     INSERT INTO Etudiant VALUES (1, 'Alice', 18.5);
5     INSERT INTO Etudiant VALUES (2, 'Bob', 12.0);
6     INSERT INTO Etudiant VALUES (3, 'Clara', 15.5);
7     INSERT INTO Etudiant VALUES (4, 'David', 9.0);
8     INSERT INTO Etudiant VALUES (5, 'Emma', 17.0);
9     INSERT INTO Etudiant VALUES (6, 'Farid', 11.5);
10    INSERT INTO Etudiant VALUES (7, 'Grace', 14.0);
11    INSERT INTO Etudiant VALUES (8, 'Hugo', 8.5);
12    INSERT INTO Etudiant VALUES (9, 'Ines', 16.0);
13    INSERT INTO Etudiant VALUES (10, 'Jamal', 13.5);
14    COMMIT;
15 END;
16 /

```

5.6 Exercice 6 — Décompte de 10 à 1 avec WHILE

```

1 DECLARE v_i NUMBER := 10;
2 BEGIN
3     WHILE v_i >= 1 LOOP
4         DBMS_OUTPUT.PUT_LINE(v_i);
5         v_i := v_i - 1;
6     END LOOP;
7 END;
8 /

```

5.7 Exercice 7 — Augmenter le prix de tous les produits de 10%

```

1 BEGIN
2     UPDATE Produit SET prix = prix * 1.10;
3     COMMIT;
4     DBMS_OUTPUT.PUT_LINE('Prix augmentes de 10%.');
5 END;
6 /

```

5.8 Exercice 8 — Supprimer les produits dont le prix < 50

```

1 DECLARE v_nb NUMBER;

```

```
2 BEGIN
3     DELETE FROM Produit WHERE prix < 50;
4     v_nb := SQL%ROWCOUNT;
5     COMMIT;
6     DBMS_OUTPUT.PUT_LINE(v_nb || ' produit(s) supprime(s).');
7 END;
8 /
```

5.9 Exercice 9 — Tester le signe d'un nombre

```
1 DECLARE v_nombre NUMBER := -7;
2 BEGIN
3     IF v_nombre > 0 THEN DBMS_OUTPUT.PUT_LINE('Positif.');
```

```
4     ELSEIF v_nombre < 0 THEN DBMS_OUTPUT.PUT_LINE('Negatif.');
```

```
5     ELSE DBMS_OUTPUT.PUT_LINE('Nul.');
```

```
6     END IF;
```

```
7 END;
```

```
8 /
```

5.10 Exercice 10 — Carrés de 1 à 20

```
1 CREATE TABLE Carre (nombre NUMBER, carre NUMBER);
2
3 BEGIN
4     FOR i IN 1..20 LOOP
5         INSERT INTO Carre VALUES(i, i*i);
6         DBMS_OUTPUT.PUT_LINE(i || '^2 = ' || (i*i));
7     END LOOP;
8     COMMIT;
9 END;
```

```
10 /
```

Chapter 6

TP PL/SQL — Curseurs

Présentation

Un curseur est une zone mémoire permettant de traiter ligne par ligne le résultat d'un SELECT. Ce TP présente les curseurs explicites (OPEN/FETCH/CLOSE) et la boucle FOR (gestion semi-automatique).

6.1 Exercice 1 — Base Usine (curseur explicite)

6.1.1 Q1 — Liste de toutes les commandes

```
1 DECLARE
2     CURSOR c_commandes IS
3         SELECT Numcmd, Datcmd, Numfour, Mncmd FROM Commande;
4     v_cmd Commande%ROWTYPE;
5 BEGIN
6     OPEN c_commandes;
7     LOOP
8         FETCH c_commandes INTO v_cmd;
9         EXIT WHEN c_commandes%NOTFOUND;
10        DBMS_OUTPUT.PUT_LINE('N' || v_cmd.Numcmd ||
11                               ' | ' || v_cmd.Numfour || ' | ' || v_cmd.Mncmd);
12    END LOOP;
13    CLOSE c_commandes;
14 END;
15 /
```

6.1.2 Q2 — Commandes dont le montant est supérieur à la moyenne

```
1 DECLARE
2     CURSOR c_sup_moy IS
3         SELECT Numcmd, Datcmd, Numfour, Mncmd FROM Commande
4         WHERE Mncmd > (SELECT AVG(Mncmd) FROM Commande);
5     v_cmd Commande%ROWTYPE;
6 BEGIN
7     OPEN c_sup_moy;
8     LOOP
9         FETCH c_sup_moy INTO v_cmd;
10        EXIT WHEN c_sup_moy%NOTFOUND;
11        DBMS_OUTPUT.PUT_LINE('N' || v_cmd.Numcmd || ' | ' ||
12                               v_cmd.Mncmd);
13    END LOOP;
14    CLOSE c_sup_moy;
15 END;
16 /
```

6.1.3 Q3 — Commandes avec fournisseur parisien

```

1 DECLARE
2     CURSOR c_paris IS
3         SELECT C.Numcmd, C.Numfour, C.Mncmd
4         FROM   Commande C
5         JOIN   Fournisseurs F ON C.Numfour = F.Numfour
6         WHERE  F.Adrfour = 'Paris';
7     v_numcmd  Commande.Numcmd%TYPE;
8     v_numfour Commande.Numfour%TYPE;
9     v_mncmd   Commande.Mncmd%TYPE;
10 BEGIN
11     OPEN c_paris;
12     LOOP
13         FETCH c_paris INTO v_numcmd, v_numfour, v_mncmd;
14         EXIT WHEN c_paris%NOTFOUND;
15         DBMS_OUTPUT.PUT_LINE('N' || v_numcmd || ' | ' || v_numfour);
16     END LOOP;
17     CLOSE c_paris;
18 END;
19 /

```

6.2 Exercice 2 — Base Gavasoft (boucle FOR)

6.2.1 Q1 — Employés avec commission, par ordre décroissant

```

1 DECLARE
2     CURSOR c_comm IS
3         SELECT NomE, Comm FROM Emp
4         WHERE  Comm IS NOT NULL ORDER BY Comm DESC;
5 BEGIN
6     FOR emp IN c_comm LOOP
7         DBMS_OUTPUT.PUT_LINE(emp.NomE || ' | ' || emp.Comm);
8     END LOOP;
9 END;
10 /

```

6.2.2 Q2 — Embauches depuis le 01/09/2006

```

1 DECLARE
2     CURSOR c_emb IS
3         SELECT NomE, Embauche FROM Emp
4         WHERE  Embauche >= TO_DATE('01/09/2006','DD/MM/YYYY');
5 BEGIN
6     FOR emp IN c_emb LOOP
7         DBMS_OUTPUT.PUT_LINE(emp.NomE || ' | ' || emp.Embauche);
8     END LOOP;
9 END;
10 /

```

6.2.3 Q3 — Employés travaillant à Créteil

```

1 DECLARE
2     CURSOR c_creteil IS
3         SELECT E.NomE, D.Lieu FROM Emp E
4         JOIN   Dept D ON E.NumD = D.NumD WHERE D.Lieu = 'Creteil';
5 BEGIN

```

```
6      FOR emp IN c_creteil LOOP
7          DBMS_OUTPUT.PUT_LINE(emp.NomE || ' | ' || emp.Lieu);
8      END LOOP;
9  END;
10 /
```

6.2.4 Q4 — Subordonnés de Guimezanes

```
1  DECLARE
2      CURSOR c_sub IS
3          SELECT NomE, Fonction FROM Emp
4          WHERE NumS = (SELECT NumE FROM Emp WHERE NomE = 'Guimezanes');
5  BEGIN
6      FOR emp IN c_sub LOOP
7          DBMS_OUTPUT.PUT_LINE(emp.NomE || ' | ' || emp.Fonction);
8      END LOOP;
9  END;
10 /
```

6.2.5 Q5 — Moyenne des salaires

```
1  DECLARE
2      CURSOR c_sal IS SELECT Salaire FROM Emp;
3      v_total NUMBER := 0; v_nb NUMBER := 0;
4  BEGIN
5      FOR emp IN c_sal LOOP
6          v_total := v_total + emp.Salaire; v_nb := v_nb + 1;
7      END LOOP;
8      DBMS_OUTPUT.PUT_LINE('Moyenne : ' || ROUND(v_total/v_nb, 2));
9  END;
10 /
```

6.2.6 Q6 — Nombre de commissions non NULL

```
1  DECLARE
2      CURSOR c_comm IS SELECT Comm FROM Emp WHERE Comm IS NOT NULL;
3      v_nb NUMBER := 0;
4  BEGIN
5      FOR emp IN c_comm LOOP v_nb := v_nb + 1; END LOOP;
6      DBMS_OUTPUT.PUT_LINE('Commissions non NULL : ' || v_nb);
7  END;
8  /
```

6.2.7 Q7 — Employés gagnant plus que la moyenne

```
1  DECLARE
2      v_moy NUMBER;
3      CURSOR c_plus IS
4          SELECT NomE, Salaire FROM Emp WHERE Salaire > v_moy;
5  BEGIN
6      SELECT AVG(Salaire) INTO v_moy FROM Emp;
7      FOR emp IN c_plus LOOP
8          DBMS_OUTPUT.PUT_LINE(emp.NomE || ' | ' || emp.Salaire);
9      END LOOP;
10 END;
11 /
```

Chapter 7

TP — Procédures Stockées et Fonctions

Présentation

Ce TP porte sur la création de procédures stockées et de fonctions PL/SQL : modes de paramètres (IN, OUT, IN OUT), manipulation de tables et gestion des exceptions.

7.1 Exercice 1 — Premier contact

```
1  -- Bloc anonyme
2  BEGIN DBMS_OUTPUT.PUT_LINE('Hello'); END;
3  /
4
5  -- Fonction carree
6  CREATE OR REPLACE FUNCTION carre(p_n IN NUMBER) RETURN NUMBER IS
7  BEGIN RETURN p_n * p_n; END;
8  /
9  SELECT carre(7) FROM DUAL; -- 49
```

7.2 Exercice 2 — Maximum de deux nombres

```
1  CREATE OR REPLACE PROCEDURE max_deux(
2      p_a IN NUMBER, p_b IN NUMBER, p_max OUT NUMBER) IS
3  BEGIN
4      IF p_a >= p_b THEN p_max := p_a; ELSE p_max := p_b; END IF;
5  END;
6  /
7
8  DECLARE v_r NUMBER;
9  BEGIN max_deux(15,42,v_r); DBMS_OUTPUT.PUT_LINE('Max : ' || v_r); END;
10 /
```

7.3 Exercice 3 — Permutation

```
1  CREATE OR REPLACE PROCEDURE permuter(p_a IN OUT NUMBER, p_b IN OUT
2      NUMBER) IS
3  BEGIN v_tmp := p_a; p_a := p_b; p_b := v_tmp; END;
4  /
```

7.4 Exercice 4 — Augmenter le salaire

```
1  CREATE OR REPLACE PROCEDURE augmenter_salaire(
```

```

2      p_nom IN VARCHAR2, p_taux IN NUMBER) IS
3      v_nb NUMBER;
4  BEGIN
5      SELECT COUNT(*) INTO v_nb FROM EMPLOYES WHERE ENAME = p_nom;
6      IF v_nb = 0 THEN DBMS_OUTPUT.PUT_LINE('Employe introuvable. ');
7      ELSIF v_nb > 1 THEN DBMS_OUTPUT.PUT_LINE('Nom non unique. ');
8      ELSE
9          UPDATE EMPLOYES SET SALARY = SALARY*(1+p_taux/100) WHERE
              ENAME=p_nom;
10         COMMIT;
11     END IF;
12 END;
13 /
14 EXEC augmenter_salaire('Ali', 10);

```

7.5 Exercice 5 — Conversion Dirhams vers Euros

```

1  CREATE OR REPLACE FUNCTION convertir_euros(
2      p_montant IN NUMBER, p_taux IN NUMBER DEFAULT 11)
3  RETURN NUMBER IS
4  BEGIN RETURN ROUND(p_montant / p_taux, 2); END;
5  /
6  SELECT convertir_euros(1100) FROM DUAL; -- 100

```

7.6 Exercice 6 — Employés au-dessus d'un seuil

```

1  CREATE OR REPLACE PROCEDURE afficher_salaires_sup(
2      p_seuil IN NUMBER, p_ordre IN VARCHAR2 DEFAULT 'ASC') IS
3  BEGIN
4      IF p_seuil < 0 THEN
5          DBMS_OUTPUT.PUT_LINE('Montant negatif invalide. '); RETURN;
6      END IF;
7      FOR emp IN (SELECT ENAME, SALARY FROM EMPLOYES WHERE SALARY >
9          p_seuil
10             ORDER BY CASE WHEN p_ordre='DESC' THEN SALARY END DESC,
11                 CASE WHEN p_ordre!='DESC' THEN SALARY END ASC)
12     LOOP
13         DBMS_OUTPUT.PUT_LINE(emp.ENAME || ' | ' || emp.SALARY);
14     END LOOP;
15 END;
16 /
17 EXEC afficher_salaires_sup(7000, 'DESC');

```

7.7 Exercice 7 — Salaire moyen par département

```

1  CREATE OR REPLACE FUNCTION salaire_moyen_dept(p_dept IN VARCHAR2)
2  RETURN NUMBER IS
3      v_moy NUMBER;
4  BEGIN
5      SELECT AVG(SALARY) INTO v_moy FROM EMPLOYES WHERE DEPT = p_dept;
6      RETURN ROUND(NVL(v_moy, 0), 2);
7  EXCEPTION WHEN NO_DATA_FOUND THEN RETURN NULL;
8  END;
9  /

```

```
10 SELECT salaire_moyen_dept('D1') FROM DUAL;
```

7.8 Exercice 8 — Mise à jour d'adresse

```
1 CREATE OR REPLACE PROCEDURE maj_adresse(
2   p_nom IN VARCHAR2, p_nouvelle IN VARCHAR2) IS
3 BEGIN
4   IF TRIM(p_nouvelle) IS NULL THEN
5     DBMS_OUTPUT.PUT_LINE('Adresse vide refusee.');
```

```
6   RETURN;
7   UPDATE EMPLOYES SET ADDRESS = p_nouvelle
8   WHERE ENAME = p_nom AND ROWNUM = 1;
9   IF SQL%ROWCOUNT = 0 THEN DBMS_OUTPUT.PUT_LINE('Employe
10     introuvable.');
```

```
11   ELSE COMMIT; END IF;
12 END;
13 /
```

7.9 Exercice 9 — Nombre d'employés par département

```
1 CREATE OR REPLACE FUNCTION nb_employes_dept(p_dept IN VARCHAR2)
2 RETURN NUMBER IS
3   v_nb NUMBER;
4 BEGIN
5   SELECT COUNT(*) INTO v_nb FROM EMPLOYES WHERE DEPT = p_dept;
6   RETURN v_nb;
7 END;
8 /
9
10 CREATE OR REPLACE PROCEDURE bilan_departements IS
11 BEGIN
12   FOR d IN (SELECT DISTINCT DEPT FROM EMPLOYES ORDER BY DEPT) LOOP
13     DBMS_OUTPUT.PUT_LINE('Dept ' || d.DEPT ||
14       ' : ' || nb_employes_dept(d.DEPT) || ' employe(s)' ||
15       ' | Moy : ' || salaire_moyen_dept(d.DEPT));
16   END LOOP;
17 END;
18 /
```

7.10 Exercice 10 — Suppression des petits salaires

```
1 CREATE OR REPLACE PROCEDURE supprimer_petits_salaires(p_seuil IN
2   NUMBER) IS
3   v_nb NUMBER;
4 BEGIN
5   SAVEPOINT avant_suppression;
6   SELECT COUNT(*) INTO v_nb FROM EMPLOYES WHERE SALARY < p_seuil;
7   IF v_nb = 0 THEN
8     DBMS_OUTPUT.PUT_LINE('Aucun employe sous le seuil.');
```

```
9   RETURN;
10   END IF;
11   FOR emp IN (SELECT ENAME, SALARY FROM EMPLOYES WHERE SALARY < p_seuil)
12   LOOP
13     DBMS_OUTPUT.PUT_LINE('Supprime : ' || emp.ENAME);
14   END LOOP;
```

```
12      DELETE FROM EMPLOYES WHERE SALARY < p_seuil;
13      COMMIT;
14  EXCEPTION
15      WHEN OTHERS THEN
16          ROLLBACK TO avant_suppression;
17          DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);
18  END;
19  /
20  EXEC supprimer_petits_salaires(5000);
```

7.11 Synthèse — Procédure vs Fonction

Critère	Procédure	Fonction
Retourne une valeur	Non (via OUT seulement)	Oui (RETURN)
Appel dans une requête SQL	Non	Oui
Transaction possible	Oui	Déconseillé
Utilisation typique	Actions, mises à jour	Calculs, conversions

Chapter 8

TP — Triggers PL/SQL

Présentation

Un trigger est un bloc PL/SQL déclenché automatiquement lors d'un événement DML (INSERT, UPDATE, DELETE). Ce TP implémente 9 règles métier complexes ainsi que 4 exercices supplémentaires sur une base de gestion universitaire.

8.1 Création des tables

```
1 DROP TABLE passer      CASCADE CONSTRAINTS;
2 DROP TABLE examen      CASCADE CONSTRAINTS;
3 DROP TABLE inscription CASCADE CONSTRAINTS;
4 DROP TABLE prerequis   CASCADE CONSTRAINTS;
5 DROP TABLE module      CASCADE CONSTRAINTS;
6 DROP TABLE etudiant    CASCADE CONSTRAINTS;
7
8 CREATE TABLE etudiant (
9     id_etudiant NUMBER PRIMARY KEY, nom VARCHAR2(50),
10    prenom VARCHAR2(50), date_inscription DATE, moyenne NUMBER);
11
12 CREATE TABLE module (
13     code_module VARCHAR2(10) PRIMARY KEY, intitule VARCHAR2(100),
14     effecMax NUMBER, note_min NUMBER);
15
16 CREATE TABLE prerequis (
17     module VARCHAR2(10) REFERENCES module(code_module),
18     prerequis VARCHAR2(10) REFERENCES module(code_module),
19     note_min NUMBER, PRIMARY KEY(module, prerequis));
20
21 CREATE TABLE inscription (
22     id_etudiant NUMBER REFERENCES etudiant(id_etudiant),
23     code_module VARCHAR2(10) REFERENCES module(code_module),
24     date_inscription DATE, PRIMARY KEY(id_etudiant, code_module));
25
26 CREATE TABLE examen (
27     id_examen NUMBER PRIMARY KEY,
28     code_module VARCHAR2(10) REFERENCES module(code_module),
29     date_examen DATE, coefficient NUMBER);
30
31 CREATE TABLE passer (
32     id_etudiant NUMBER REFERENCES etudiant(id_etudiant),
33     id_examen NUMBER REFERENCES examen(id_examen),
34     note NUMBER, PRIMARY KEY(id_etudiant, id_examen));
```

8.2 Tableau des règles et codes d'erreur

Règle	Code	Signification
1	-20001	Modification de note_min interdite
2	-20002	Dépassement de effecMax
3	-20003	Création examen sans inscrits
4	-20004	Date examen antérieure à l'inscription
5	-20005	Circuit dans les prérequis
6	-20006	Prérequis non satisfaits
7	—	Mise à jour automatique de la moyenne
8	-20008	Modification note_min invalide des inscriptions
9	-20009	Réduction effecMax en dessous du nombre d'inscrits

8.3 Règle 1 — Interdiction de modifier note_min dans prerequis

```

1 CREATE OR REPLACE TRIGGER trg_prerequis_no_update_note_min
2 BEFORE UPDATE OF note_min ON prerequis
3 FOR EACH ROW
4 BEGIN
5     RAISE_APPLICATION_ERROR(-20001,
6         'Modification de note_min dans prerequis interdite.');
```

8.4 Règle 2 — Contrôle de effecMax lors d'une inscription

```

1 CREATE OR REPLACE TRIGGER trg_inscription_check_effectif
2 BEFORE INSERT ON inscription FOR EACH ROW
3 DECLARE
4     v_inscrits NUMBER; v_effecMax NUMBER;
5 BEGIN
6     SELECT effecMax INTO v_effecMax FROM module
7     WHERE code_module = :NEW.code_module;
8     SELECT COUNT(*) INTO v_inscrits FROM inscription
9     WHERE code_module = :NEW.code_module;
10    IF v_inscrits >= v_effecMax THEN
11        RAISE_APPLICATION_ERROR(-20002,
12            'Module ' || :NEW.code_module || ' : effectif maximum
13            atteint.');
```

8.5 Règle 3 — Examen uniquement si le module a des inscrits

```

1 CREATE OR REPLACE TRIGGER trg_examen_check_inscrits
2 BEFORE INSERT ON examen FOR EACH ROW
3 DECLARE v_nb NUMBER;
4 BEGIN
5     SELECT COUNT(*) INTO v_nb FROM inscription
6     WHERE code_module = :NEW.code_module;
7     IF v_nb = 0 THEN
```

```

8      RAISE_APPLICATION_ERROR(-20003,
9          'Aucun inscrit pour le module ' || :NEW.code_module);
10  END IF;
11  END;
12  /

```

8.6 Règle 4 — Date d'examen postérieure à la date d'inscription

```

1  CREATE OR REPLACE TRIGGER trg_passer_check_date_inscription
2  BEFORE INSERT ON passer FOR EACH ROW
3  DECLARE
4      v_date_insc DATE;  v_date_exam DATE;  v_module VARCHAR2(10);
5  BEGIN
6      SELECT code_module, date_examen INTO v_module, v_date_exam
7      FROM examen WHERE id_examen = :NEW.id_examen;
8      SELECT date_inscription INTO v_date_insc
9      FROM inscription WHERE id_etudiant = :NEW.id_etudiant
10     AND code_module = v_module;
11     IF v_date_insc >= v_date_exam THEN
12         RAISE_APPLICATION_ERROR(-20004,
13             'Inscription posterieure a la date d''examen.');
14     END IF;
15 EXCEPTION WHEN NO_DATA_FOUND THEN
16     RAISE_APPLICATION_ERROR(-20004, 'Etudiant non inscrit au module. ');
17 END;
18 /

```

8.7 Règle 5 — Détection de circuit dans les prérequis

```

1  CREATE OR REPLACE FUNCTION existe_circuit(
2      p_module VARCHAR2, p_cible VARCHAR2) RETURN BOOLEAN IS
3      CURSOR c_enfants IS
4          SELECT prerequis FROM prerequis WHERE module = p_module;
5  BEGIN
6      FOR enfant IN c_enfants LOOP
7          IF enfant.prerequis = p_cible THEN RETURN TRUE; END IF;
8          IF existe_circuit(enfant.prerequis, p_cible) THEN RETURN TRUE;
9              END IF;
10     END LOOP;
11     RETURN FALSE;
12 END;
13 /
14 CREATE OR REPLACE TRIGGER trg_prerequis_no_circuit
15 BEFORE INSERT OR UPDATE ON prerequis FOR EACH ROW
16 BEGIN
17     IF existe_circuit(:NEW.prerequis, :NEW.module) THEN
18         RAISE_APPLICATION_ERROR(-20005,
19             'Circuit detecte : ' || :NEW.module || ' -> ' ||
20             :NEW.prerequis);
21     END IF;
22 END;
23 /

```

8.8 Règle 6 — Vérification des prérequis à l'inscription

```

1 CREATE OR REPLACE TRIGGER trg_inscription_check_prerequis
2 BEFORE INSERT ON inscription FOR EACH ROW
3 DECLARE
4     CURSOR c_prereq IS
5         SELECT p.prerequis, p.note_min FROM prerequis p
6         WHERE p.module = :NEW.code_module;
7     v_note_obtenue NUMBER;
8 BEGIN
9     FOR prereq IN c_prereq LOOP
10        BEGIN
11            SELECT MAX(pa.note) INTO v_note_obtenue
12            FROM passer pa JOIN examen e ON pa.id_examen = e.id_examen
13            WHERE pa.id_etudiant = :NEW.id_etudiant
14            AND e.code_module = prereq.prerequis;
15        EXCEPTION WHEN NO_DATA_FOUND THEN v_note_obtenue := NULL; END;
16        IF v_note_obtenue IS NULL OR v_note_obtenue < prereq.note_min
17            THEN
18            RAISE_APPLICATION_ERROR(-20006, 'Prerequis ' ||
19                prereq.prerequis || ' non satisfait. Note requise : ' ||
20                prereq.note_min);
21        END IF;
22    END LOOP;
23 END;
/
```

8.9 Règle 7 — Mise à jour automatique de la moyenne

```

1 CREATE OR REPLACE TRIGGER trg_passer_update_moyenne
2 AFTER INSERT OR UPDATE OR DELETE ON passer FOR EACH ROW
3 DECLARE
4     v_id_etudiant NUMBER; v_moyenne NUMBER; v_compteur NUMBER;
5 BEGIN
6     IF INSERTING OR UPDATING THEN v_id_etudiant := :NEW.id_etudiant;
7     ELSE v_id_etudiant := :OLD.id_etudiant; END IF;
8     SELECT SUM(pa.note * e.coefficient) / NULLIF(SUM(e.coefficient),0),
9         COUNT(*) INTO v_moyenne, v_compteur
10    FROM passer pa JOIN examen e ON pa.id_examen = e.id_examen
11    WHERE pa.id_etudiant = v_id_etudiant AND pa.note IS NOT NULL;
12    IF v_compteur > 0 THEN
13        UPDATE etudiant SET moyenne = ROUND(v_moyenne,2)
14        WHERE id_etudiant = v_id_etudiant;
15    ELSE
16        UPDATE etudiant SET moyenne = NULL WHERE id_etudiant =
17            v_id_etudiant;
18    END IF;
19 END;
/
```

8.10 Règle 8 — Modification de note_min si elle invalide des inscriptions

```

1 CREATE OR REPLACE TRIGGER trg_prerequis_check_note_min_update
2 BEFORE UPDATE OF note_min ON prerequis FOR EACH ROW
```

```

3 DECLARE
4     CURSOR c_etudiants IS
5         SELECT DISTINCT i.id_etudiant FROM inscription i
6         WHERE i.code_module = :NEW.module;
7     v_note_max NUMBER;
8 BEGIN
9     FOR etu IN c_etudiants LOOP
10        SELECT MAX(pa.note) INTO v_note_max
11        FROM passer pa JOIN examen e ON pa.id_examen = e.id_examen
12        WHERE pa.id_etudiant = etu.id_etudiant
13        AND e.code_module = :NEW.prerequis;
14        IF v_note_max IS NULL OR v_note_max < :NEW.note_min THEN
15            RAISE_APPLICATION_ERROR(-20008,
16                'Nouvelle note_min invaliderait l''etudiant ' ||
17                etu.id_etudiant);
18        END IF;
19    END LOOP;
20 END;
21 /

```

8.11 Règle 9 — Modification de effecMax sans invalider des inscriptions

```

1 CREATE OR REPLACE TRIGGER trg_module_check_effecMax_update
2 BEFORE UPDATE OF effecMax ON module FOR EACH ROW
3 DECLARE v_inscrits NUMBER;
4 BEGIN
5     SELECT COUNT(*) INTO v_inscrits FROM inscription
6     WHERE code_module = :NEW.code_module;
7     IF :NEW.effecMax < v_inscrits THEN
8         RAISE_APPLICATION_ERROR(-20009,
9             'Nouvel effectif max ( ' || :NEW.effecMax ||
10             ') inferieur au nombre d''inscrits ( ' || v_inscrits || ')');
11     END IF;
12 END;
13 /

```

8.12 Exercices supplémentaires

8.12.1 Contrainte 10 — Module déjà validé

```

1 CREATE OR REPLACE TRIGGER trg_inscription_deja_valide
2 BEFORE INSERT ON inscription FOR EACH ROW
3 DECLARE v_note_max NUMBER; v_note_min NUMBER;
4 BEGIN
5     SELECT note_min INTO v_note_min FROM module
6     WHERE code_module = :NEW.code_module;
7     SELECT MAX(pa.note) INTO v_note_max
8     FROM passer pa JOIN examen e ON pa.id_examen = e.id_examen
9     WHERE pa.id_etudiant = :NEW.id_etudiant
10    AND e.code_module = :NEW.code_module;
11    IF v_note_max IS NOT NULL AND v_note_max >= v_note_min THEN
12        RAISE_APPLICATION_ERROR(-20010,
13            'Module deja valide avec note ' || v_note_max);
14    END IF;

```

```

15 EXCEPTION WHEN NO_DATA_FOUND THEN NULL;
16 END;
17 /

```

8.12.2 Contrainte 11 — Conflit de planning (7 jours)

```

1 CREATE OR REPLACE TRIGGER trg_examen_conflit_planning
2 BEFORE INSERT ON examen FOR EACH ROW
3 DECLARE v_nb NUMBER;
4 BEGIN
5     SELECT COUNT(*) INTO v_nb FROM examen
6     WHERE code_module = :NEW.code_module
7     AND ABS(:NEW.date_examen - date_examen) <= 7;
8     IF v_nb > 0 THEN
9         RAISE_APPLICATION_ERROR(-20011,
10             'Examen en conflit de planning pour ' || :NEW.code_module);
11     END IF;
12 END;
13 /

```

8.12.3 Contrainte 12 — Moyenne calculée seulement si tous les examens sont passés

```

1 CREATE TABLE log_avis (
2     id_log NUMBER GENERATED ALWAYS AS IDENTITY,
3     id_etudiant NUMBER, message VARCHAR2(200), date_log DATE DEFAULT
4     SYSDATE);
5
6 CREATE OR REPLACE TRIGGER trg_passer_moyenne_complete
7 AFTER INSERT OR UPDATE OR DELETE ON passer FOR EACH ROW
8 DECLARE
9     v_id_etudiant NUMBER; v_nb_inscrits NUMBER;
10    v_nb_passes NUMBER; v_moyenne NUMBER;
11    PRAGMA AUTONOMOUS_TRANSACTION;
12 BEGIN
13     IF INSERTING OR UPDATING THEN v_id_etudiant := :NEW.id_etudiant;
14     ELSE v_id_etudiant := :OLD.id_etudiant; END IF;
15     SELECT COUNT(DISTINCT e.id_examen) INTO v_nb_inscrits
16     FROM inscription i JOIN examen e ON i.code_module = e.code_module
17     WHERE i.id_etudiant = v_id_etudiant;
18     SELECT COUNT(*) INTO v_nb_passes
19     FROM passer WHERE id_etudiant = v_id_etudiant AND note IS NOT NULL;
20     IF v_nb_passes < v_nb_inscrits THEN
21         UPDATE etudiant SET moyenne = NULL WHERE id_etudiant =
22             v_id_etudiant;
23         INSERT INTO log_avis(id_etudiant, message) VALUES(
24             v_id_etudiant,
25             'Moyenne non calculée : ' || v_nb_passes || '/' ||
26             v_nb_inscrits || ' examens passés. ');
27         COMMIT;
28     ELSE
29         SELECT SUM(pa.note*e.coefficient)/NULLIF(SUM(e.coefficient),0)
30         INTO v_moyenne FROM passer pa
31         JOIN examen e ON pa.id_examen = e.id_examen
32         WHERE pa.id_etudiant = v_id_etudiant;
33         UPDATE etudiant SET moyenne = ROUND(v_moyenne,2)
34         WHERE id_etudiant = v_id_etudiant;

```

```
33         COMMIT;
34     END IF;
35 END;
36 /
```

8.12.4 Contrainte 13 — Interdire la suppression d'un module prérequis

```
1 CREATE OR REPLACE TRIGGER trg_module_no_delete_si_prerequis
2 BEFORE DELETE ON module FOR EACH ROW
3 DECLARE v_nb NUMBER;
4 BEGIN
5     SELECT COUNT(*) INTO v_nb FROM prerequis WHERE prerequis =
6         :OLD.code_module;
7     IF v_nb > 0 THEN
8         RAISE_APPLICATION_ERROR(-20013,
9             'Impossible de supprimer ' || :OLD.code_module ||
10             ' : utilise comme prerequis par ' || v_nb || ' module(s).');
11     END IF;
12 END;
13 /
```

Conclusion

Ce rapport regroupe l'ensemble des travaux pratiques et dirigés réalisés dans le cadre du module **Bases de Données Avancées** sous la direction du Professeur H. Hrimech.

Les thèmes abordés couvrent un spectre complet des compétences attendues :

- **SQL avancé** : sous-requêtes simples, synchronisées, multi-colonnes, quantificateurs ANY / ALL.
- **Vues** : création, WITH CHECK OPTION, WITH READ ONLY, vues matérialisées, sécurité et optimisation.
- **Index** : création, unicité, suppression.
- **PL/SQL fondamental** : variables, %TYPE, %ROWTYPE, RECORD, collections, structures de contrôle.
- **Curseurs** : explicites (OPEN/FETCH/CLOSE) et implicites (boucle FOR).
- **Procédures et fonctions** : modes IN/OUT/IN OUT, gestion des exceptions, transactions.
- **Triggers** : contraintes complexes, détection de circuits, mises à jour en cascade, transactions autonomes.

La maîtrise de ces concepts constitue le socle indispensable pour le développement d'applications robustes et sécurisées sur Oracle.